Project Design and Requirements Document

Single-Room Hybrid AI Energy Management System

Energy Waste and Anomaly Detection

Minimum Viable Product (MVP) Design

Core Design Principle

Local rule-based control for reliability, data-driven anomaly detection for intelligence, and policy-validated automatic actions for safety.

Project Type:	Embedded IoT and AI-based Energy Management
Initial Deployment:	Single-room pilot
Controller:	ESP8266-based local controller
Initial AI Feature:	Energy Waste and Anomaly Detection
Document Version:	1.0 – Initial Design Plan
Date:	29 October 2026

This document describes the proposed system design. Implementation status and measured performance must be verified through testing.

Contents

1 Executive Summary **5**

2 Introduction **5**

2.1 Background 5

2.2 Problem Statement 6

2.3 Proposed Solution 6

3 Project Objectives **6**

4 Scope of the MVP **7**

4.1 Included in the Initial Version 7

4.2 Not Included as a Guaranteed MVP Capability 8

5 Existing Prototype and Hardware Assumptions **8**

5.1 Hardware Verification Requirements 9

6 System Architecture **9**

6.1 Architectural Overview 9

6.2 Local Controller 10

6.3 Backend Service 10

6.4 Recommended Initial Technology Stack 11

7 Hybrid Control Strategy **11**

7.1 Local Rule-Based Control 11

7.2 Backend Intelligence 12

7.3 Policy-Validated Automatic Actions 12

8 Functional Requirements **12**

9 Data Model and Schema **13**

9.1 Sensor Samples 13

9.2 Device State Events 14

9.3 Anomaly Events 15

9.4 Control Actions 16
9.5 Device Policies and Manual Overrides 16

10 Example Device Policy **16**

11 Anomaly Detection Logic **17**

11.1 Detection A: Possible Vacancy with Device ON 17
11.2 Detection B: Daylight-Related Lighting Waste 18
11.3 Detection C: Unusually Long Runtime 18
11.4 Detection D: Rapid ON/OFF Cycling 19
11.5 Detection E: Invalid or Missing Sensor Data 19

12 Initial Threshold Policy **20**

12.1 Device-Specific Action Policy 20

13 Control Policy Validation **21**

13.1 Action Decision Examples 21

14 Severity and Alert Management **22**

15 Communication and Event Processing **23**

15.1 Initial Reporting Intervals 23
15.2 Message Reliability 23
15.3 Offline Operation 24

16 Non-Functional Requirements **24**

16.1 Reliability 24
16.2 Safety 24
16.3 Maintainability 25
16.4 Privacy and Security 25

17 Implementation Roadmap **25**

17.1 Phase 1: Event Logging 25
17.2 Phase 2: Rule-Based Detection 26
17.3 Phase 3: Limited Automatic Control 26

17.4 Phase 4: Historical Baseline 26
17.5 Phase 5: Evaluation and Refinement 27

18 Testing and Evaluation Plan **27**
18.1 Functional Test Cases 27
18.2 Performance Metrics 28
18.3 Runtime Comparison 29

19 Energy Measurement Limitations **29**

20 Risks and Mitigation **30**

21 Future Enhancements **30**

22 Expected Deliverables **31**

23 System Terminology and Definitions **32**
23.1 Internet of Things (IoT) 32
23.2 Embedded Controller 32
23.3 Hybrid Control Strategy 32
23.4 Rule-Based Automation 32
23.5 Energy Waste Detection 33
23.6 Anomaly Detection 33
23.7 Occupancy Inference 33
23.8 Historical Baseline 33
23.9 Policy Validation 34
23.10 Event Logging 34
23.11 Runtime-Based Energy Indicator 34
23.12 Minimum Viable Product (MVP) 34

24 Technology Stack and Development Tools **35**
24.1 Hardware Technologies 35
24.2 Firmware and Embedded Development 35
24.3 Monitoring and Communication 36

24.4 Proposed Backend and Data Storage 36
24.5 Anomaly Detection Technology 36
24.6 Technology Selection Rationale 37

25 System Operational Process **37**
25.1 End-to-End Workflow 37
25.2 Step 1: Sensor Acquisition 37
25.3 Step 2: Local State Tracking 38
25.4 Step 3: Local Rule Evaluation 38
25.5 Step 4: Event Transmission and Storage 38
25.6 Step 5: Anomaly Detection 38
25.7 Step 6: Alert Generation 39
25.8 Step 7: Policy Validation 39
25.9 Step 8: Action Execution and Audit 39
25.10Step 9: Review and Improvement 40

26 Device-Specific Operating Process **40**
26.1 Lighting 40
26.2 Fan 40
26.3 Air Conditioner 40
26.4 PC and Other Equipment 41

27 Development Methodology and Implementation Process **41**
27.1 Phase 1: Existing Prototype Review 41
27.2 Phase 2: Event Logging 41
27.3 Phase 3: Rule-Based Anomaly Detection 41
27.4 Phase 4: Policy-Validated Automatic Control 41
27.5 Phase 5: Historical Baseline and Statistical Analysis 42
27.6 Phase 6: Evaluation and Refinement 42

28 System Constraints and Technical Limitations **42**

29 Conclusion **43**

1 Executive Summary

The Single-Room Hybrid AI Energy Management System is an IoT-based project intended to improve energy-use awareness, identify potential energy waste, and support safe automatic control of electrical devices in a room.

The system will initially use an ESP8266-based controller, environmental sensors, a motion sensor, a light sensor, and relay-controlled devices. The initial device categories are lighting, fans, air conditioners, and personal computers or other electrical equipment.

The first major intelligent feature will be **Energy Waste and Anomaly Detection**. It will identify conditions such as devices remaining on during a likely vacancy, lights operating when ambient daylight is sufficient, unusually long operating sessions, and abnormal ON/OFF cycling.

The proposed control strategy is hybrid:

- Local rule-based control will operate on the embedded controller.
- A backend service will store historical data and perform analytics and anomaly detection.
- The backend may propose control actions, but those actions must pass local safety and policy validation.
- Manual override will be supported.
- Local control must remain available when the internet or backend is unavailable.

The initial prototype has relay state information but no dedicated power or energy meter. Consequently, the first version can analyze operating duration and potential waste, but it cannot directly measure actual power consumption, energy consumption in kWh, or verified energy savings.

The system will be developed incrementally, beginning with reliable event logging and rule-based alerts. Limited automatic control will be enabled only after the relevant rules and safeguards have been tested.

2 Introduction

2.1 Background

Unnecessary operation of lights, fans, air conditioners, computers, and other equipment can increase electricity consumption in offices, classrooms, and other indoor spaces.

Traditional relay-based automation can switch devices on or off according to fixed conditions. However, fixed rules alone may not adequately describe normal device usage or identify unusual operating patterns.

An intelligent energy management system can improve visibility by combining sensor readings, device state history, operating duration, and contextual information.

2.2 Problem Statement

The proposed system addresses the following problems:

- Electrical devices may remain switched on when they are not needed.
- Users may not know how long individual devices have been operating.
- Unusual device usage may go unnoticed.
- Simple automation may generate false actions when sensor readings are incomplete or misleading.
- Relay state alone does not provide direct measurement of actual energy consumption.
- Automatic control must remain safe and predictable when AI services or network connectivity fail.

2.3 Proposed Solution

The system will combine embedded sensing and local automation with backend-based event analysis.

The first implementation will detect rule-defined conditions and generate alerts. Historical usage data will later be used to develop adaptive thresholds and statistical anomaly detection.

3 Project Objectives

The primary objectives are:

1. Monitor the operating states of selected electrical devices in one room.
2. Collect and timestamp sensor readings, device state changes, and control events.
3. Identify possible energy waste using vacancy, ambient light, and operating-duration information.

4. Generate understandable alerts that include evidence and the reason for detection.
5. Support automatic control only when an action is explicitly allowed by a device-specific policy.
6. Preserve local rule-based control during network or backend outages.
7. Maintain a history of anomalies, alerts, manual overrides, and automatic control decisions.
8. Build a structured dataset for future data-driven anomaly detection.
9. Establish a repeatable evaluation process for detection accuracy, false alarms, response time, and device runtime.

4 Scope of the MVP

4.1 Included in the Initial Version

The MVP will include:

- One-room deployment.
- ESP8266-based local control.
- Motion, ambient light, temperature, and humidity data where supported by the installed sensors.
- Relay-state event logging.
- Four device categories: light, fan, AC, and PC or other equipment.
- Local rule-based vacancy handling.
- Daylight-related lighting waste detection.
- Long-runtime detection.
- Basic abnormal ON/OFF cycling detection.
- Backend event storage and anomaly history.
- Alerts and a basic monitoring interface.
- Manual override and action logging.
- Policy validation before automatic actions.
- Historical baseline collection.

4.2 Not Included as a Guaranteed MVP Capability

The following capabilities are outside the guaranteed initial scope:

- Direct measurement of electrical power or kWh without a suitable energy meter.
- Guaranteed or quantified electricity savings.
- Accurate headcount estimation from a PIR sensor.
- General-purpose machine learning running on the ESP8266.
- Automatic AC temperature adjustment without a compatible AC control interface.
- Light dimming without compatible dimming hardware.
- Safe forced shutdown of PCs without a suitable graceful-shutdown integration.
- Multi-room production deployment.
- Fully autonomous control without policy limits.

These capabilities may be considered in later versions after hardware, safety, and data requirements are satisfied.

5 Existing Prototype and Hardware Assumptions

The project is based on an ESP8266-oriented IoT prototype.

The following components are considered part of the intended prototype configuration. The exact pin mapping, sensor models, relay wiring, and firmware behavior must be confirmed against the actual hardware before deployment.

Component	Intended Role
ESP8266 / NodeMCU	Local controller, sensor processing, relay commands, and network communication.
PIR motion sensor	Provides motion events used as an indicator of recent room activity. It does not measure exact occupancy.
LDR light sensor	Provides a raw light reading that can be calibrated to identify sufficient ambient daylight.
Temperature sensor	Provides room temperature data where supported.
Humidity sensor	Provides room humidity data where supported.
Relay module	Switches supported electrical loads through the appropriate relay channels.

LCD or monitoring display	Optional local display of status, readings, or alerts.
Network connection	Transfers events and receives permitted commands when the network is available.

5.1 Hardware Verification Requirements

Before connecting the system to real electrical loads, the implementation team must verify:

- Correct ESP8266 board and GPIO mapping.
- Relay channel mapping and active logic level.
- Electrical isolation and relay load ratings.
- Sensor voltage compatibility and wiring.
- Stable power supply and grounding arrangements.
- Correct startup states and reset behavior.
- Device-specific electrical protection requirements.

The system must not assume that a software command proves that a physical device has actually changed state.

6 System Architecture

6.1 Architectural Overview

The architecture separates sensing, local control, analytics, policy validation, and monitoring.

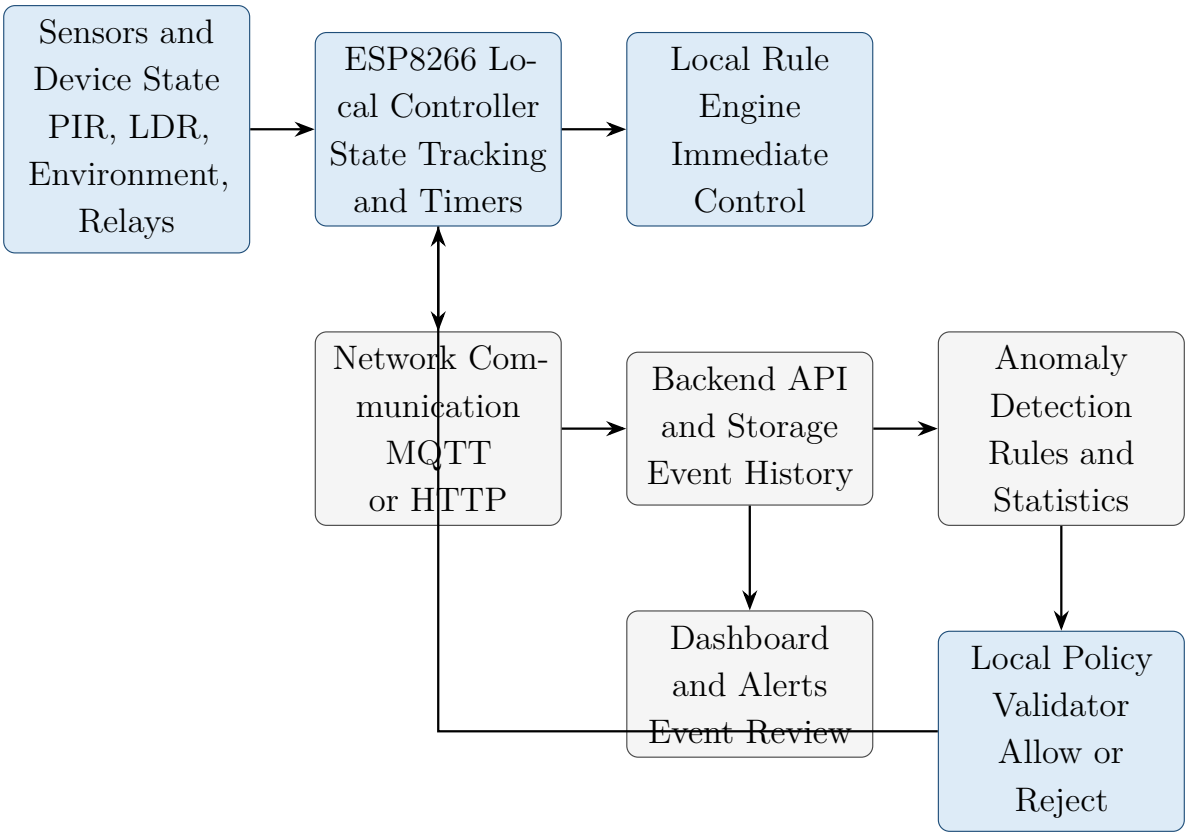


Figure 1: Proposed hybrid system architecture

6.2 Local Controller

The ESP8266 will:

- Read available sensor inputs.
- Track recent motion and device state changes.
- Run local rules that do not depend on the backend.
- Apply validated relay commands.
- Respect manual override and minimum switching intervals.
- Report events and controller status.
- Reject expired, duplicate, or disallowed commands.

6.3 Backend Service

The backend will:

- Receive and validate event messages.
- Store timestamped sensor and device events.
- Calculate operating durations.
- Evaluate rule-based and statistical detectors.
- Generate anomaly records and alerts.
- Maintain configuration and historical baselines.
- Propose control actions where permitted.

6.4 Recommended Initial Technology Stack

Layer	Proposed Technology
Embedded controller	ESP8266 firmware in C/C++ using the appropriate board libraries.
Communication	MQTT or HTTP over the local network.
Backend API	Python with FastAPI, or an equivalent lightweight service.
Initial database	SQLite for a single-room pilot.
Dashboard	A simple web dashboard or existing monitoring interface.
Alerts	Dashboard notifications first; external notifications may be added later.
Future database	PostgreSQL or another suitable database if deployment expands.

These are proposed choices, not implemented or tested components at the time of writing.

7 Hybrid Control Strategy

The system will use a hybrid strategy consisting of local rules, backend analytics, and policy-validated actions.

7.1 Local Rule-Based Control

Local rules will handle functions that must remain available even if the backend is offline.

Examples include:

- Tracking the time since the latest motion event.
- Applying configured vacancy rules.
- Enforcing minimum switching intervals.
- Respecting manual override.
- Rejecting commands that violate local policy.

7.2 Backend Intelligence

The backend will analyze historical usage and identify conditions that are difficult to detect from a single sensor reading.

Examples include:

- Operating sessions that are unusually long.
- Repeated abnormal switching patterns.
- Device usage that differs from its historical baseline.
- Repeated vacancy-related waste events.

7.3 Policy-Validated Automatic Actions

An AI or analytics service may propose an action, but the local controller must validate it before execution.

The backend must not be allowed to bypass local safety rules by sending a direct, unrestricted relay command.

If the backend is unavailable, the controller will continue supported local rules and will not invent or execute new AI-based decisions.

8 Functional Requirements

ID	Requirement
FR-01	The system shall identify each monitored device using a unique device identifier.
FR-02	The system shall record available sensor readings with timestamps.
FR-03	The system shall record each requested device state change and its source.

FR-04	The system shall calculate operating duration from recorded ON and OFF events.
FR-05	The system shall identify potential waste during a confirmed or suspected vacancy according to configured rules.
FR-06	The system shall identify potential daylight-related lighting waste after sensor calibration.
FR-07	The system shall identify configured maximum-runtime violations.
FR-08	The system shall identify unusually frequent ON/OFF transitions according to a configurable rule.
FR-09	The system shall store anomaly type, severity, evidence, and detection method.
FR-10	The system shall generate alerts for configured anomaly conditions.
FR-11	The system shall support device-specific automatic control permissions.
FR-12	The system shall support manual override.
FR-13	The system shall reject automatic actions that violate local policy.
FR-14	The system shall log the reason for every automatic action or rejected proposal.
FR-15	The system shall maintain local rule-based operation during backend or internet failure.
FR-16	The system shall identify missing or invalid sensor data and avoid unsafe actions based on invalid readings.
FR-17	The system shall support historical data analysis when sufficient records are available.
FR-18	The system shall distinguish estimated runtime-based indicators from measured energy consumption.

9 Data Model and Schema

The data model separates sensor readings, device events, anomalies, control actions, and device policies.

All timestamps should be stored in UTC. The dashboard may display timestamps in the user's local time zone.

9.1 Sensor Samples

The `sensor_samples` dataset will store sensor readings and derived room-activity state.

Field	Type	Description

sample_id	UUID or ID	Unique sample identifier.
room_id	String	Identifier for the monitored room.
timestamp_utc	Timestamp	Time the sample was recorded.
pir_motion	Boolean or nullable	Motion indication for the relevant observation interval.
ldr_raw	Integer or nullable	Raw light sensor reading.
ambient_light_level	Number or nullable	Calibrated light level, if available.
temperature_c	Number or nullable	Temperature in Celsius.
humidity_pct	Number or nullable	Relative humidity percentage.
last_motion_at	Timestamp or nullable	Time of the latest motion event.
occupancy_state	Enum	Derived room activity state.
sensor_quality	Enum	Validity or quality status.

Recommended occupancy states:

- `occupied_recently`
- `vacancy_candidate`
- `vacant_confirmed`
- `unknown`

These states represent an inference from available sensor history, not a guaranteed headcount.

9.2 Device State Events

The `device_state_events` dataset will store requested device transitions and their context.

Field	**Type**	**Description**
event_id	UUID or ID	Unique event identifier.
timestamp_utc	Timestamp	Event time.
device_id	String	Unique device identifier.
requested_state	Enum	Requested ON or OFF state.
state_source	Enum	Manual, local rule, or AI proposal.

reason_code	String	Reason for the state change.
controller_mode	Enum	AUTO or MANUAL mode.
command_status	Enum	Pending, applied, failed, or unknown.
observed_state	Enum	Confirmed state if feedback hardware exists; otherwise unknown or assumed.

Initial device identifiers:

- `light_01`
- `fan_01`
- `ac_01`
- `pc_01`

If multiple PCs or other devices are added, each must receive a separate identifier.

9.3 Anomaly Events

The `anomaly_events` dataset will store detected or suspected problems.

Field	Type	Description
anomaly_id	UUID or ID	Unique anomaly identifier.
timestamp_utc	Timestamp	Detection time.
device_id	String or nullable	Related device, if applicable.
anomaly_type	Enum	Vacancy runtime, daylight waste, long runtime, rapid cycling, or sensor issue.
severity	Enum	Info, warning, or high.
detector_type	Enum	Rule-based or statistical.
observed_value	Number or JSON	Value that triggered detection.
threshold_value	Number or JSON	Relevant configured or statistical threshold.
evidence	JSON	Supporting sensor and event information.
status	Enum	Open, acknowledged, resolved, or false positive.
resolved_at	Timestamp or nullable	Resolution time.

9.4 Control Actions

The `control_actions` dataset will provide an audit trail.

Field	Description
action_id	Unique action identifier.
timestamp_utc	Time of the proposed or executed action.
device_id	Device associated with the action.
trigger_anomaly_id	Related anomaly, if applicable.
proposed_state	Proposed ON or OFF state.
policy_decision	Allowed or blocked.
decision_reason	Explanation of the policy result.
policy_version	Version of the policy used.
execution_result	Success, failed, expired, or not executed.

9.5 Device Policies and Manual Overrides

Each device policy should contain at least:

- Device identifier.
- Whether automatic vacancy control is allowed.
- Vacancy grace period.
- Optional maximum runtime.
- Minimum ON and OFF durations.
- Whether alerts are enabled.
- Whether automatic actions are allowed.
- Policy version.

Manual override records should contain the device identifier, start time, expiry time, and optional reason.

10 Example Device Policy

The following JSON illustrates a possible fan policy. The values are examples for initial testing and must be validated before real deployment.

```
{
  "device_id": "fan_01",
  "auto_off_on_vacancy": true,
  "vacancy_action_after_min": 15,
  "max_continuous_on_min": null,
  "min_on_time_min": 2,
  "min_off_time_min": 1,
  "alert_enabled": true,
  "auto_action_enabled": true,
  "policy_version": 1
}
```

A null maximum runtime means that no maximum runtime has yet been configured.

Device policies must be independently configurable. A fan policy must not automatically be reused for an air conditioner or a personal computer.

11 Anomaly Detection Logic

The MVP will initially use deterministic rules. Statistical methods will be added after sufficient historical data has been collected.

11.1 Detection A: Possible Vacancy with Device ON

Purpose: Identify devices that remain on while the room appears to be unused.

Inputs:

- Time of the latest motion event.
- Current inferred occupancy state.
- Device relay state.
- Device-specific vacancy policy.
- Manual override state.

Logic:

1. Track the time of the latest PIR motion event.
2. If no motion has been detected for the configured initial period, set the room state to `vacancy_candidate`.

3. If the longer configured threshold is reached, re-evaluate the sensor state and device state.
4. If a device remains ON and the conditions persist, create a vacancy-related anomaly.
5. Generate an alert.
6. If automatic action is permitted for that device, submit an action to local policy validation.

The absence of PIR motion does not prove that the room is empty. Stationary occupants may not trigger a PIR sensor. The vacancy rule must therefore include a grace period and must be evaluated for false shutdowns.

11.2 Detection B: Daylight-Related Lighting Waste

Purpose: Identify cases where a light remains ON despite sufficient ambient daylight.

Conditions:

- The light relay is ON.
- The calibrated light reading exceeds the configured daylight threshold.
- The condition persists for the required confirmation interval.
- The system is not under a relevant manual override.

If the conditions remain true for the configured confirmation period, create a `daylight_waste` anomaly and generate an alert.

The LDR threshold must be calibrated in the actual room. Raw analog readings must not be treated as universal lux values without calibration.

Hysteresis and minimum switching intervals should be used to avoid repeated ON/OFF transitions when the light reading is close to the threshold.

11.3 Detection C: Unusually Long Runtime

For a device session with a recorded ON event followed by an OFF event, calculate:

$$T_{\text{runtime}} = t_{\text{OFF}} - t_{\text{ON}}$$

If a configured maximum runtime is exceeded, create a long-runtime anomaly.

If no OFF event has occurred, calculate the current session duration using the current time. Such a session must be identified as still open rather than assumed to have ended.

The system will support two threshold types:

1. **Configured threshold:** A manually specified maximum duration.
2. **Adaptive threshold:** A threshold derived from comparable historical operating sessions.

Adaptive thresholds should be enabled only when enough valid historical records are available. An initial criterion of at least 20 comparable sessions may be used for evaluation, but this is a configurable starting point rather than a universal statistical requirement.

11.4 Detection D: Rapid ON/OFF Cycling

Purpose: Detect excessive device state changes within a configured time window.

For initial testing, a rule such as more than four state transitions in ten minutes may be evaluated.

The threshold must be calibrated for each device. Manual transitions and expected automation events should be recorded separately where possible.

Air conditioners require particular care because frequent power cycling may be inappropriate for the equipment. The system must not use rapid-cycling detection as permission to repeatedly switch an AC relay.

11.5 Detection E: Invalid or Missing Sensor Data

If required sensor data is missing, stale, or outside the configured valid range:

- Record a sensor-quality event.
- Avoid high-confidence vacancy conclusions.
- Do not execute actions that depend on invalid data.
- Continue independent local rules that remain safe.
- Report the data-quality issue to the monitoring layer.

12 Initial Threshold Policy

The following values are proposed starting points for controlled testing. They are not final operational settings and must be adjusted using observed room behavior.

Condition	Initial Value	Purpose
No detected motion	5 minutes	Mark vacancy as a candidate, not as certain vacancy.
Extended vacancy	10 minutes	Potentially generate a warning if a device remains ON.
Vacancy action threshold	15 minutes	Allow re-evaluation for eligible devices before a permitted automatic action.
Daylight confirmation	3 minutes	Require a persistent light reading before creating an alert.
Rapid cycling example	More than 4 transitions in 10 minutes	Generate a diagnostic warning after device-specific validation.
Historical baseline	At least 20 comparable sessions as an initial criterion	Evaluate whether a statistical runtime baseline is sufficiently supported.

The initial vacancy periods are separate stages of one proposed policy. They must be implemented with consistent timing and must not cause repeated commands.

For example, a warning generated at ten minutes should not create a new duplicate warning every evaluation cycle. A cooldown or deduplication mechanism should be used.

12.1 Device-Specific Action Policy

Device	Detection Policy	Initial Automatic Action Policy
Light	Vacancy, daylight, and long runtime.	OFF may be permitted after re-evaluation and policy validation.
Fan	Vacancy, long runtime, and rapid cycling.	OFF may be permitted after vacancy confirmation, override checks, and minimum-time checks.

AC	Vacancy, long runtime, and unusual cycling.	Alert-only initially. Later automation requires a compatible control interface and suitable safeguards.
PC or other equipment	Vacancy and long runtime.	Alert-only initially. Forced shutdown is not allowed without an appropriate graceful-shutdown mechanism and validated integration.

The policy should be conservative. A device's long runtime alone does not prove that the device is unnecessarily operating.

13 Control Policy Validation

Every proposed automatic action must pass the local policy validator.

The validator should check:

1. The device is included in the automatic-action allowlist.
2. The controller is in AUTO mode.
3. No active manual override blocks the action.
4. Required sensor data is valid and sufficiently recent.
5. The device's minimum ON/OFF interval has been met.
6. The command is not duplicated or expired.
7. The proposed transition is valid for the current device state.
8. The relevant device-specific safeguards are met.

If any required condition fails, the action must be rejected and the reason recorded.

13.1 Action Decision Examples

Condition	**Decision**	**Reason**

Fan is ON, vacancy rule passes, AUTO mode is active, and no override exists.	Potentially allow OFF.	All configured conditions must still pass at execution.
Light is ON, but the LDR reading is invalid.	Reject automatic action.	Daylight evidence is unreliable.
PC is ON and a vacancy anomaly is detected.	Alert only.	A forced relay shutdown may cause data loss.
AC has reached a runtime threshold.	Alert only in the initial version.	Runtime alone does not justify a power-cycle action.
Manual override is active.	Block ordinary AI actions.	The user's manual choice must be respected.
Backend is offline.	Continue local rules.	No new backend-dependent AI decision is available.

14 Severity and Alert Management

The MVP will use a small set of understandable severity levels.

Severity	**Example**	**Response**
Info	Unusual runtime without sufficient evidence of waste.	Record the event.
Warning	Persistent vacancy-related or daylight-related condition.	Show a warning and retain supporting evidence.
High	A persistent potential waste condition confirmed by the applicable rules.	Generate a prominent alert and evaluate any policy-authorized action.
Diagnostic	Missing or invalid sensor data, communication failure, or inconsistent event history.	Log the fault and suppress unsafe decisions.

Alert deduplication and cooldown must prevent the same unresolved condition from producing excessive repeated notifications.

Each alert should contain:

- Device identifier.
- Detection time.
- Anomaly type and severity.

- Observed condition.
- Threshold that was exceeded.
- Recommended response.
- Whether an automatic action was proposed, allowed, blocked, or executed.

15 Communication and Event Processing

The controller and backend must exchange timestamped messages using a defined message structure.

15.1 Initial Reporting Intervals

The following intervals are proposed for initial testing:

- Sensor sample: approximately every 30 seconds.
- Motion and device state transition: event-driven.
- Controller heartbeat: approximately every 30 seconds.
- Anomaly evaluation: on relevant events or every 30–60 seconds.

These intervals should be adjusted according to network reliability, sensor behavior, memory use, and the needs of the final implementation.

15.2 Message Reliability

Each event should have a unique identifier to support deduplication after network reconnection.

Commands should include:

- Unique action identifier.
- Device identifier.
- Requested state.
- Issue time and expiry time.
- Reason or trigger identifier.
- Relevant policy version.

The controller should reject expired commands and duplicate action identifiers.

15.3 Offline Operation

When the network or backend is unavailable:

- Local rule-based control must continue where safe.
- Backend-dependent AI actions must not be executed.
- The controller should buffer important events if persistent storage is available.
- Buffered events should be transmitted after reconnection.
- If an event cannot be retained, the system should not claim that the event was permanently stored.

Persistent event buffering must account for ESP8266 memory limitations and flash write endurance. The implementation should avoid unnecessary frequent writes to non-volatile storage.

16 Non-Functional Requirements

16.1 Reliability

- Local control must not depend on continuous internet access.
- Invalid sensor data must not trigger unsafe automatic actions.
- The controller must start in a safe, defined state.
- Communication failures must be handled explicitly.

16.2 Safety

- Device-specific action permissions are required.
- Manual override must be respected.
- Minimum switching intervals must be enforced.
- AC and PC control must remain conservative until appropriate interfaces and safeguards are validated.
- Electrical isolation, relay ratings, enclosures, and wiring protection must be appropriate for the load.

16.3 Maintainability

- Thresholds must be configurable rather than scattered as unexplained constants.
- Each detector must have a clear identifier.
- Policy changes must be versioned.
- Logs must explain why each decision was made.
- Device identifiers must be consistent across firmware, backend, and dashboard.

16.4 Privacy and Security

- Only required sensor and operational data should be collected.
- Communication credentials must not be exposed in public source code.
- Backend endpoints should authenticate control requests where supported.
- Control commands must be validated before use.
- Network access should be restricted to the intended devices and services.

17 Implementation Roadmap

Development will proceed in stages. Each stage should be tested before the next stage is enabled.

17.1 Phase 1: Event Logging

Tasks:

- Define device identifiers and event formats.
- Record sensor readings and relay state changes.
- Timestamp events consistently.
- Store events in the backend database.
- Track device ON sessions and their durations.

Exit criteria: Device events and sensor records can be inspected and used to calculate operating duration.

17.2 Phase 2: Rule-Based Detection

Tasks:

- Implement vacancy candidate and confirmation logic.
- Implement daylight-related detection after LDR calibration.
- Implement configured long-runtime rules.
- Implement rapid-cycling diagnostics.
- Generate alerts without automatic control initially.

Exit criteria: Test cases produce expected anomaly records, and false alarms can be reviewed.

17.3 Phase 3: Limited Automatic Control

Tasks:

- Implement a local action allowlist.
- Implement manual override and expiry.
- Implement minimum ON/OFF intervals.
- Implement duplicate-command protection.
- Enable eligible Light and Fan actions only after controlled testing.
- Keep AC and PC alert-only in the initial version.

Exit criteria: Every allowed or rejected action has an auditable decision reason.

17.4 Phase 4: Historical Baseline

Tasks:

- Collect valid device operating sessions.
- Separate manual and automatic state changes.
- Compare operating duration across comparable time periods.
- Introduce robust statistical thresholds.
- Evaluate false positives before enabling adaptive alerts.

Exit criteria: The baseline is based on adequate data and its limitations are documented.

17.5 Phase 5: Evaluation and Refinement

Tasks:

- Evaluate detection accuracy.
- Review false alarms and missed events.
- Measure detection latency.
- Compare runtime against an appropriate baseline.
- Refine device-specific thresholds.
- Document the results and remaining limitations.

18 Testing and Evaluation Plan

18.1 Functional Test Cases

Test ID	Test Scenario	Expected Result
TC-01	Device changes from OFF to ON.	A timestamped state event is recorded.
TC-02	Device changes from ON to OFF.	The completed runtime session is calculated.
TC-03	No PIR motion is detected for the configured vacancy period while an eligible device is ON.	A vacancy-related anomaly or alert is generated according to the policy.
TC-04	Ambient daylight remains above the calibrated threshold while the light is ON.	A daylight-related alert is generated after the confirmation interval.
TC-05	A device exceeds its configured maximum runtime.	A long-runtime anomaly is recorded.
TC-06	A device exceeds its rapid-cycling threshold.	A diagnostic anomaly is generated.
TC-07	Manual override is active when an AI action is proposed.	The action is blocked and the reason is logged.
TC-08	Required sensor data is missing or invalid.	Unsafe actions based on that data are suppressed.
TC-09	Backend or internet connection is lost.	Local supported rules continue to operate.

TC-10	An expired or duplicate control command is received.	The controller rejects the command.
TC-11	An AC or PC vacancy anomaly is detected in the initial policy configuration.	An alert is generated without an unauthorized forced shutdown.
TC-12	An automatic action is allowed and executed.	The proposed state, policy decision, execution result, and reason are logged.

18.2 Performance Metrics

The following metrics will be used.

Precision:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall:

$$\text{Recall} = \frac{TP}{TP + FN}$$

where:

- TP: correctly detected waste or anomaly events.
- FP: false positive detections.
- FN: real events that were not detected.

These metrics require a reviewed set of test events or a manually labeled evaluation dataset. An alert count alone does not establish detection quality.

Additional metrics include:

- Detection latency.
- False alerts per day.
- Missed events during controlled tests.
- Successful and rejected automatic actions.
- Device operating duration.
- Percentage of events successfully stored.
- Frequency of sensor or communication failures.

18.3 Runtime Comparison

Runtime reduction can be estimated by comparing device operating duration over comparable observation periods.

The comparison should account for room usage, device availability, manual operation, and changes in occupancy.

A reduction in runtime does not necessarily prove a reduction in electricity consumption because device power varies and may not be known.

19 Energy Measurement Limitations

The current system records relay states but does not have a dedicated power or energy meter.

Consequently, it can calculate:

- Device operating duration.
- Number of ON sessions.
- Number of potential waste events.
- Estimated runtime reduction.

It cannot directly determine:

- Actual instantaneous power in watts.
- Actual energy consumption in kWh.
- Verified monetary savings.
- Verified percentage reduction in electrical energy.

If a nominal device power rating is available, a rough estimate may be calculated:

$$E_{\text{estimated}} = P_{\text{rated}} \times T_{\text{runtime}}$$

where power is expressed in kW and runtime in hours.

This estimate assumes that the device operates at its rated power for the entire session. That assumption may be inaccurate, especially for air conditioners, computers, and variable-speed equipment.

A future version should add a suitable power or energy meter to measure real electrical consumption. Any savings evaluation should then compare measured energy over comparable baseline and intervention periods.

20 Risks and Mitigation

Risk	Impact	Mitigation
PIR does not detect a stationary occupant.	False vacancy and unwanted shutdown.	Use grace periods, confirmation checks, and conservative device policies.
LDR readings are not calibrated.	False daylight-related alerts.	Calibrate the sensor in the actual room and use confirmation intervals.
Relay command is mistaken for confirmed device state.	Incorrect runtime or action reporting.	Separate requested state from observed state and report unknown feedback when no feedback hardware exists.
Backend becomes unavailable.	Historical analysis and remote alerts are interrupted.	Maintain local rules and use an appropriate event buffer.
False anomaly alerts.	Users may ignore notifications.	Use deduplication, severity levels, and reviewed thresholds.
Unsafe PC or AC shutdown.	Data loss or equipment stress.	Keep these devices alert-only until suitable interfaces and safeguards are validated.
Incorrect device power assumptions.	Misleading energy-saving claims.	Label runtime-based estimates and add a suitable energy meter for measured kWh.
Electrical wiring or load mismatch.	Equipment damage or safety hazards.	Verify relay ratings, isolation, wiring, and protection before connecting real loads.

21 Future Enhancements

After the MVP has been evaluated, future versions may consider:

- Adding a power or energy meter.
- Developing a richer device usage baseline.

- Adding statistical or machine-learning anomaly detection.
- Predicting room occupancy from historical activity data.
- Optimizing operation using occupancy and environmental context.
- Adding compatible dimming or variable-speed control hardware.
- Integrating a supported AC control interface.
- Expanding the architecture to multiple rooms.
- Adding role-based access and richer reporting.
- Comparing measured energy consumption against a defined baseline.

These are future possibilities, not guaranteed features of the initial implementation.

22 Expected Deliverables

The MVP is expected to produce the following deliverables:

1. Documented embedded-controller behavior.
2. A consistent device and event identifier scheme.
3. Sensor and device event logging.
4. A backend data store for operational history.
5. Rule-based anomaly detection.
6. Anomaly and alert records with supporting evidence.
7. Device-specific control policies.
8. Manual override and policy-validation logic.
9. A basic dashboard or event review interface.
10. A set of functional test cases and results.
11. An evaluation report covering detection quality, false alerts, response time, and runtime.
12. A documented list of hardware limitations and future improvements.

23 System Terminology and Definitions

This section defines the major technical terms used in the proposed Single-Room Hybrid AI Energy Management System. These definitions establish a common understanding of the system architecture, operational behavior, and intelligent energy management process.

23.1 Internet of Things (IoT)

The Internet of Things (IoT) refers to a network of physical devices equipped with sensors, controllers, and communication capabilities. In this project, the IoT layer consists of the ESP8266 controller, environmental sensors, motion sensor, light sensor, relay module, and supported network communication.

The IoT layer collects room-related information and enables monitoring and control of connected electrical devices.

23.2 Embedded Controller

An embedded controller is a microcontroller-based device that performs dedicated control and processing tasks. The proposed system uses an ESP8266-based NodeMCU board for sensor acquisition, local rule execution, relay control, and communication with the monitoring or analytics layer.

23.3 Hybrid Control Strategy

Hybrid control combines local rule-based automation with backend-based data analysis and intelligent decision support.

In this project, local rules handle essential control functions and remain available during network or backend failure. The analytics layer identifies unusual operating patterns and may propose actions. Every proposed automatic action must pass the applicable local policy and safety checks before execution.

23.4 Rule-Based Automation

Rule-based automation uses predefined conditions and thresholds to determine system behavior.

For example, if no motion is detected for a configured period and a permitted device remains ON, the system may generate a warning or propose an automatic OFF action.

Rule-based automation will be the foundation of the initial Minimum Viable Product (MVP).

23.5 Energy Waste Detection

Energy waste detection identifies operating conditions that may indicate unnecessary use of electrical devices.

Examples include:

- A light or fan remaining ON during a prolonged period of likely room vacancy.
- A light remaining ON despite sufficient ambient daylight.
- A device operating longer than its configured or historically expected duration.

These conditions indicate potential waste rather than guaranteed energy loss. Additional context may be required to determine whether the device operation was necessary.

23.6 Anomaly Detection

Anomaly detection identifies events or patterns that differ from configured rules or expected historical behavior.

The initial implementation will use deterministic rules. After sufficient valid historical data is collected, statistical methods may be introduced to identify unusually long operating sessions or abnormal switching patterns.

The project will not claim machine-learning performance until a suitable model has been implemented and evaluated.

23.7 Occupancy Inference

Occupancy inference estimates whether a room is likely to be occupied using available information, particularly PIR motion events and their timestamps.

A PIR sensor detects motion rather than the exact number of people. Therefore, the absence of a recent motion event does not prove that the room is empty.

The system will use configurable grace periods and confirmation checks before taking vacancy-related action.

23.8 Historical Baseline

A historical baseline represents the normal operating pattern estimated from previously recorded device and sensor events.

The baseline may include operating duration, usage time, and the frequency of device state changes.

The baseline will initially be established from collected operational data. Statistical thresholds will be enabled only when sufficient valid and comparable observations are available.

23.9 Policy Validation

Policy validation is the process of checking whether a proposed control action is permitted by the device's configuration and safety constraints.

The validation process checks factors such as automatic control permission, manual override, sensor data quality, minimum switching intervals, and command validity.

A proposed AI action must not bypass the local policy validator.

23.10 Event Logging

Event logging is the process of recording timestamped system activities, including sensor readings, device state changes, anomaly detections, alerts, manual overrides, and control decisions.

Event logs provide the historical data required for debugging, performance evaluation, and future data-driven analysis.

23.11 Runtime-Based Energy Indicator

Runtime-based indicators describe how long a device has been operating and how often it has been switched ON or OFF.

Because the initial prototype does not include a dedicated power or energy meter, runtime-based analysis will be used as an indicator of potential waste.

It will not be presented as direct measurement of electrical power, energy consumption in kWh, or verified energy savings.

23.12 Minimum Viable Product (MVP)

The MVP is the first practical version of the system that implements the essential monitoring, event logging, rule-based anomaly detection, alert generation, and policy-validated control functions.

Advanced predictive optimization and broader multi-room deployment will be considered future enhancements.

24 Technology Stack and Development Tools

The project will use an incremental technology strategy. The existing prototype components will be retained where suitable, while additional software services will be introduced only when required by the MVP.

24.1 Hardware Technologies

The intended hardware configuration is summarized below. Exact component models, pin assignments, electrical ratings, and wiring must be confirmed against the actual prototype before implementation or deployment.

- **ESP8266 / NodeMCU:** The primary embedded controller for sensor acquisition, local automation, relay commands, and communication.
- **PIR Motion Sensor:** Provides motion events used to estimate recent room activity.
- **LDR Light Sensor:** Provides ambient-light readings for daylight-related lighting analysis after calibration.
- **Temperature and Humidity Sensor:** Provides environmental readings where supported by the installed sensor module.
- **Relay Module:** Switches supported electrical loads according to validated control commands.
- **LCD Display:** May provide local status information, sensor readings, or system messages if retained in the implementation.
- **Electrical Devices:** The initial device categories are lighting, fans, air conditioners, and PCs or other equipment.

24.2 Firmware and Embedded Development

The ESP8266 firmware will be developed using the Arduino development environment and C/C++-based Arduino sketches, subject to confirmation of the existing development setup.

The firmware will be responsible for:

- Reading available sensor inputs.
- Tracking motion events and device state changes.
- Maintaining local operating timers.

- Executing supported rule-based control.
- Enforcing manual override and switching constraints.
- Transmitting operational events when connected.
- Rejecting invalid or unauthorized control commands.

24.3 Monitoring and Communication

The existing prototype is associated with the Blynk IoT platform. Where compatible with the required functionality, Blynk may be retained for device status display and initial monitoring.

The project will distinguish between the monitoring interface and the analytics engine. Dashboard availability alone does not provide historical anomaly detection or guarantee that all required event data is persistently stored.

If the existing platform cannot support the required data processing and historical storage, a separate backend service will be introduced.

24.4 Proposed Backend and Data Storage

The following technologies are proposed for the MVP backend, subject to final implementation decisions:

- **Python:** Backend logic and data analysis.
- **FastAPI:** A lightweight API service for receiving events and providing analysis results.
- **SQLite:** Initial database for timestamped sensor records, device state events, anomalies, control actions, and configuration.
- **HTTP REST API:** A possible communication interface between the controller, backend, and monitoring services, depending on the final integration design.

These are proposed technologies rather than a statement that the backend has already been implemented. The communication method and integration with the existing Blynk setup will be finalized before coding.

24.5 Anomaly Detection Technology

The initial detection engine will use configurable rules and thresholds. Examples include vacancy-related runtime, daylight-related lighting waste, long runtime, and rapid ON/OFF cycling.

After sufficient historical data is available, the system may introduce statistical methods such as median and median absolute deviation (MAD) to identify operating sessions that are unusually long compared with comparable historical sessions.

A machine-learning model is not required for the first rule-based MVP. Any future model must be evaluated using appropriate test data before being used to influence automatic control.

24.6 Technology Selection Rationale

The proposed stack is intended to:

- Reuse the existing embedded prototype where possible.
- Keep essential control functions local.
- Separate sensor control from historical analytics.
- Support structured event storage and traceable decisions.
- Allow incremental development within a single-room scope.
- Provide a pathway for future expansion without requiring a complex distributed architecture at the start.

25 System Operational Process

25.1 End-to-End Workflow

The proposed operational workflow is:

Sensor Acquisition → Data Validation → Local Control and Event Logging → Data Transmission → Historical Storage → Anomaly Detection → Alert Generation → Policy Validation → Permitted Action and Audit Logging

This workflow separates detection from control. Detecting an anomaly does not automatically authorize the system to switch a device OFF.

25.2 Step 1: Sensor Acquisition

The ESP8266 collects available readings from the PIR, LDR, and environmental sensors. Device state transitions and controller events are also timestamped.

Sensor readings will be checked for missing, stale, or invalid values before being used in decisions.

25.3 Step 2: Local State Tracking

The controller maintains the latest available sensor state, recent motion time, device relay state, operating timers, and manual or automatic control mode.

The system must distinguish a requested relay state from confirmed physical device feedback. Without appropriate feedback hardware, the physical state cannot be guaranteed from a software command alone.

25.4 Step 3: Local Rule Evaluation

The local rule engine evaluates configured conditions, such as prolonged absence of detected motion or minimum switching intervals.

Rules that must remain available during network failure will be executed locally. Rules dependent on historical backend analysis may generate a proposal but must not bypass local control constraints.

25.5 Step 4: Event Transmission and Storage

When communication is available, the system transmits relevant sensor readings, state transitions, and control events to the monitoring or backend service.

The backend stores valid records with unique identifiers and timestamps. Duplicate events should be handled to prevent incorrect operating-duration calculations.

If persistent buffering is implemented, selected events may be retained locally during temporary network outages and transmitted after reconnection.

25.6 Step 5: Anomaly Detection

The detection engine evaluates stored or newly received events against the configured rules.

The initial anomaly categories are:

- Potential waste during a likely room vacancy.
- Potential daylight-related lighting waste.
- Unusually long device operating sessions.
- Abnormal frequency of ON/OFF transitions.
- Sensor or communication data-quality problems.

The engine records the detected condition, evidence, threshold, detection time, and severity.

25.7 Step 6: Alert Generation

When a configured anomaly is detected, the system generates an alert containing the affected device, detection time, anomaly type, supporting evidence, and recommended response.

Alert deduplication and cooldown rules will reduce repeated notifications for the same unresolved event.

25.8 Step 7: Policy Validation

If an automatic action is proposed, the local controller checks the applicable policy before execution.

The validation process includes:

- Automatic action permission for the device.
- Current control mode and manual override state.
- Validity and freshness of required sensor data.
- Minimum ON/OFF duration constraints.
- Command expiry and duplicate-command checks.
- Device-specific safety restrictions.

If any required condition fails, the action is rejected and the decision reason is recorded.

25.9 Step 8: Action Execution and Audit

For an approved action, the controller issues the appropriate relay command and records the result.

The system logs whether the action was allowed, blocked, executed, failed, or expired. If physical feedback is unavailable, the system must not report an assumed relay state as independently confirmed device operation.

25.10 Step 9: Review and Improvement

Historical records will be reviewed to assess detection quality, false alerts, missed events, operating duration, and system reliability.

Thresholds may be adjusted based on observed behavior. Statistical detection will be introduced only when sufficient reliable historical data is available.

26 Device-Specific Operating Process

The control policy will differ by device type because the consequences of switching devices OFF are not identical.

26.1 Lighting

The system may identify possible lighting waste when the room appears vacant or when calibrated ambient daylight remains sufficient while the light is ON.

An automatic OFF action may be enabled after confirmation, manual-override checks, and policy validation.

Light dimming is outside the capabilities of a simple ON/OFF relay and requires compatible dimming hardware.

26.2 Fan

The system may identify potential waste when a fan remains ON during a prolonged period of likely vacancy.

An automatic OFF action may be enabled after the configured vacancy period and safety checks have passed. The system must account for possible PIR false-absence conditions.

26.3 Air Conditioner

The initial implementation will focus on anomaly detection and alerts for the AC.

Automatic AC control will remain restricted until a compatible control interface and appropriate operating safeguards have been confirmed. Runtime alone is not sufficient evidence to justify frequent power cycling.

26.4 PC and Other Equipment

The initial policy will use alerts for PCs and other equipment unless a safe, device-appropriate control method is available.

A forced relay shutdown must not be used as a substitute for graceful computer shutdown. Unsaved work, file corruption, and other operational risks must be considered.

27 Development Methodology and Implementation Process

The project will follow an incremental prototyping methodology. Each phase will be implemented and tested before the next phase is enabled.

27.1 Phase 1: Existing Prototype Review

The team will confirm the installed hardware, sensor models, relay mapping, firmware behavior, available monitoring functions, and electrical safety requirements.

The purpose is to establish the verified baseline of the existing prototype before extending it.

27.2 Phase 2: Event Logging

Timestamped sensor readings, device state changes, operating sessions, and control events will be recorded.

The team will verify that operating durations can be calculated correctly and that missing or duplicate events are handled appropriately.

27.3 Phase 3: Rule-Based Anomaly Detection

Vacancy-related detection, daylight-related lighting detection, long-runtime rules, and rapid-cycling diagnostics will be implemented.

The first version will operate in alert-only mode so that false alerts and missed events can be reviewed before automatic control is enabled.

27.4 Phase 4: Policy-Validated Automatic Control

After the relevant rules have been tested, limited automatic OFF actions may be enabled for approved devices, initially focusing on lighting and fans.

Manual override, minimum switching intervals, command validation, and audit logging must be verified first.

The initial policy will keep AC and PC automatic shutdown restricted unless suitable interfaces and safeguards are validated.

27.5 Phase 5: Historical Baseline and Statistical Analysis

The system will collect comparable operating sessions and establish device-specific historical baselines.

Statistical anomaly detection may then be introduced and evaluated against reviewed events. Historical thresholds must not be treated as reliable when data quality or sample size is insufficient.

27.6 Phase 6: Evaluation and Refinement

The completed MVP will be evaluated using functional tests and operational metrics, including:

- Detection precision and recall where labeled evaluation data is available.
- False alerts and missed events.
- Detection latency.
- Successful, rejected, and failed control actions.
- Device runtime and runtime reduction.
- Sensor and communication reliability.

Measured energy savings will not be claimed without suitable energy measurement hardware and a defined comparison baseline.

28 System Constraints and Technical Limitations

The following constraints will guide the initial implementation:

- The initial deployment is limited to one room.
- The controller has limited memory and processing capacity; complex analytics will therefore be assigned to a backend service if required.

- PIR motion events do not provide exact headcount and may not detect stationary occupants.
- LDR readings require calibration for reliable daylight-related decisions.
- Relay state does not directly measure actual power consumption or prove physical device operation.
- The system has no dedicated power or energy meter in the initial configuration.
- Automatic actions must be restricted by device-specific safety policies.
- Backend and network failures must not disable essential local rules.
- Proposed technologies and thresholds must be confirmed and evaluated during implementation.

These constraints are intended to keep the project technically feasible and to prevent unsupported claims about AI capability or energy savings.

29 Conclusion

The proposed Single-Room Hybrid AI Energy Management System will begin with Energy Waste and Anomaly Detection as its first major intelligent feature.

The MVP combines local rule-based control with backend analytics. It will monitor device state history, identify potential vacancy-related waste, detect daylight-related lighting waste, and flag unusually long or abnormal operating patterns.

The system will not treat every anomaly as proof of energy waste, nor will it permit unrestricted AI control. All automatic actions must follow explicit device policies, respect manual override, and pass local validation.

Because the current prototype has relay state information but no power meter, the first version will focus on operating duration, potential waste, and detection quality. Actual electrical energy consumption and verified savings will require suitable energy measurement hardware.

The recommended implementation sequence is to establish reliable event logging, validate rule-based detection in alert-only mode, introduce limited automatic control for eligible devices, collect historical data, and then add adaptive anomaly detection.

This staged approach provides a practical foundation for a safer, more intelligent, and potentially expandable energy management system.